

Section 1: Database Multi-Tenancy — The Security Proof

1.1 RLS Policy — Agent Can Only Read Their Own Leads

The following policy locks SELECT access on the leads table so that each authenticated agent sees only rows where they are the owner:

```
-- Enable RLS on the table (idempotent)
ALTER TABLE leads ENABLE ROW LEVEL SECURITY;

-- Read policy
CREATE POLICY "agents_read_own_leads"
  ON leads
  FOR SELECT
  TO authenticated
  USING (
    agent_id = auth.uid()
  );
```

How it works: `auth.uid()` resolves to the JWT sub of the currently logged-in Supabase user. Postgres evaluates the USING expression as a WHERE clause appended to every SELECT. Rows that fail the predicate are silently filtered out — no error is thrown, which prevents enumeration attacks.

1.2 RLS Policy — Agent Can Only Update Their Own Leads

For UPDATE we need both USING (filter which rows are visible to the UPDATE) and WITH CHECK (validate the proposed new state). Using an EXISTS subquery makes the ownership check explicit and resilient to future schema changes:

```
CREATE POLICY "agents_update_own_leads"
  ON leads
  FOR UPDATE
  TO authenticated
  USING (
    -- Row must already belong to the requesting agent
    EXISTS (
      SELECT 1 FROM agents
      WHERE agents.id = leads.agent_id
            AND agents.user_id = auth.uid()
    )
  )
  WITH CHECK (
    -- The row must STILL belong to the same agent after the write
```

```

    EXISTS (
      SELECT 1 FROM agents
      WHERE agents.id = leads.agent_id
            AND agents.user_id = auth.uid()
    )
  );

```

1.3 USING vs WITH CHECK — Why the Distinction Matters

Clause	When it fires	What it prevents
USING	Before the write — filters which rows can be targeted	Agent A patching Agent B's rows (row visibility)
WITH CHECK	After the write — validates the proposed new row state	Agent A reassigning a lead to Agent B by changing agent_id (data integrity after write)

Without WITH CHECK, an agent could UPDATE a lead they own and set agent_id to another agent's ID — effectively stealing the lead. USING alone only guards the targeting; WITH CHECK guards the resulting state. Both clauses are required for a complete, secure UPDATE policy.

Section 2: Resilience & Error Handling — The Scalability Proof

2.1 Retry Mechanism for 429 / 503 Errors

When processing 500 leads through an external AI API, transient failures must be handled gracefully without losing progress or hammering the API further. My design uses exponential backoff with jitter, a dead-letter queue for exhausted retries, and per-lead state tracking in the database.

Retry Algorithm

```

const MAX_RETRIES = 5;
const BASE_DELAY_MS = 1000; // 1 second

async function callWithRetry(leadId, payload, attempt = 0) {
  try {
    const res = await aiApi.process(payload);
    await db.leads.update({ id: leadId, status: 'completed', result: res });
  } catch (err) {

```

```

const isRetryable = err.status === 429 || err.status === 503;
if (!isRetryable || attempt >= MAX_RETRIES) {
  await db.leads.update({ id: leadId, status: 'failed', error: err.message });
  return;
}
// Exponential backoff with full jitter
const cap = BASE_DELAY_MS * Math.pow(2, attempt);
const delay = Math.random() * cap; // [0, cap]

// Honour Retry-After header if provided (429 often includes it)
const retryAfter = err.headers?.['retry-after'];
const wait = retryAfter ? parseInt(retryAfter) * 1000 : delay;

await db.leads.update({ id: leadId, status: 'retrying', attempts: attempt +
1 });
await sleep(wait);
return callWithRetry(leadId, payload, attempt + 1);
}
}

```

2.2 Batch State Storage Architecture

State is never held in local memory — a server crash must not lose progress. All state lives in the database so any worker can resume a batch:

Field	Type / Values	Purpose
status	pending processing retrying completed failed	Current state of each lead in the batch
attempts	integer, default 0	How many API calls have been made for this lead
next_retry_at	timestampz, nullable	When the next retry is scheduled (NULL = not yet queued)
error	text, nullable	Last error message for observability and alerting
batch_id	uuid FK → batches.id	Groups leads into a logical batch for progress reporting

```

-- Supabase schema for batch state tracking
CREATE TABLE batch_jobs (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  batch_id    uuid NOT NULL REFERENCES batches(id),
  lead_id     uuid NOT NULL REFERENCES leads(id),
  status      text NOT NULL DEFAULT 'pending'
              CHECK (status IN
('pending', 'processing', 'retrying', 'completed', 'failed')),
  attempts    int  NOT NULL DEFAULT 0,

```

```
next_retry    timestampz,  
error         text,  
updated_at    timestampz DEFAULT now()  
);  
  
-- A background worker polls for due retries:  
SELECT * FROM batch_jobs  
WHERE status = 'retrying'  
      AND next_retry <= now()  
ORDER BY next_retry  
LIMIT 50; -- process in small slices to stay within API rate limits
```

Why the database (not an in-memory queue)? For a Supabase-based SaaS, the database is the natural source of truth. Storing retry state in Postgres means: (1) any stateless serverless function can pick up work, (2) progress survives cold starts and crashes, (3) Supabase Realtime can push batch progress to the client UI with zero extra infrastructure.

Section 3: The “Worst-Case” Analysis — The Grit Proof

The Incident: Silent Insert Failures Due to Schema-Code Mismatch in OfferHive

While building OfferHive, my two-sided marketplace platform, I introduced a bug where newly created listings were silently not being saved to the database at all — yet the frontend showed no error to the user.

3.1 Root Cause

During an iterative development session I added new NOT NULL columns to the table . However, the application code that performed the insert was never updated to include these new fields in the payload.

Supabase returned a PostgreSQL NOT NULL constraint violation error, but because the error was only console.log'd and not thrown or surfaced to the UI. The insert never reached the database.

3.2 Detection

- First signal: During manual testing I created a entry, new entry was not there showing on UI.
- I initially suspected a fetch or RLS issue — I checked the frontend query and the RLS policies, both looked correct.
- Opening Vercel I checked the logs and found out where the issue is actually happening

3.3 Immediate Fix & Preventive Measures

The insertion function was updated and type safety was added to ensure it does not happen again.